Kyiv School of Economics

Computer Science Department

Software Engineering & Business Analysis

Bachelor's Thesis

MovieCrush

Movie Tracking Platform with Personalized Recommendations, Detailed Ratings, and Social Features

Anastasiia Syvak

Supervisor
KSE, Denys Zavhorodnii, dzavhorodnii@kse.org.ua

Expert
KSE, Expert Name, expert@domain.ua

Submission date
16 June 2026

TODOS

1.4 Boxes 11

- Personnal todo before marking this thesis as final

Academic Integrity Statement

I, undersigned, hereby declare that this capstone project is the result of my own work.

- All ideas, data, figures and text from other authors have been clearly cited and listed in the bibliography.
- No part of this project has been submitted previously for academic credit in this or any other institution.
- All code, diagrams, and third-party materials are either my original work or are used with permission and properly referenced.
- I have not engaged in plagiarism or any form of academic dishonesty.
- Any assistance received (e.g. from peers, tutors, or online forums) is acknowledged in the acknowledgements section.

I understand that failure to comply with these declarations constitutes academic misconduct and may lead to disciplinary action.

Place, date	Kyiv, 02.06.2026
Signature	_____

Contents

Acknowledgements 1
Abstract 3
1 Introduction 4
1.1 Basic markup 4
1.2 Images 5
1.3 Tables 5
1.4 Boxes 5
1.5 Citations, Acronyms and Glossary 6
1.6 Code 7
1.7 Context Problem 7
1.8 Objectives 8
1.9 Structure of this report 8
2 Research 9
2.1 Section 1 10
2.2 Section 2 10
2.3 Conclusion 10
3 Design 11
3.1 Architecture Overview and Requirements Alignment 11
3.1.1 Functional Requirements 11
3.1.2 Non-Functional Requirements 12
3.2 System Architecture and Major Decisions 13
3.2.1 Modular Monolith Architecture Decision 13
3.3 System Context and External Interactions 13
3.3.1 Main External Integrations 14
3.3.2 Mobile Users 15
3.4 Container Architecture and Service Decomposition 15
3.4.1 Backend Modules 15
3.4.2 Data Storage Layers 16
3.5 Technology Stack Selection 17
3.5.1 Frontend - Mobile Application 17
3.5.2 Backend 17
3.5.3 CF Service - Collaborative Filtering 18
3.5.4 Database 19
3.5.5 Infrastructure and Deployment 19
3.6 Cross-cutting Concerns 20
3.6.1 Security 20
3.6.2 Monitoring and Observability 21
3.6.3 Error Handling 21
3.6.4 Data Consistency 21
3.7 Database Design 22
3.7.1 Schema Overview 22
3.7.2 Storage Estimation Methodology 22

3.7.3 Key Design Decisions 22
3.8 User Growth Projections 23
3.9 Capacity Planning and Load Estimation 24
4 Implementation 25
4.1 Section 1 26
4.2 Section 2 26
4.3 Conclusion 26
5 Validation 27
5.1 Section 1 28
5.2 Section 2 28
5.3 Conclusion 28
6 Conclusion 29
6.1 Project summary 29
6.2 Comparison with the initial objectives 29
6.3 Encountered difficulties 29
6.4 Future perspectives 29
Glossary 30

Acknowledgements

First of all, I want to thank my parents. For giving me the opportunity to study at such a great university, for their support every single time, for always believing in me when I didn't believe in myself - they always knew I would make it. They have always been my biggest role models in life, and everything I have now is because of them. I also want to thank my mom for being someone I can tell absolutely everything to - every secret, every worry, every thought. And I want to thank my dad for teaching me tennis when I was a child - that's what led me to become the head of the KSE Table Tennis Club and find amazing friends at university, who made these student years the best ones.

I want to thank my grandparents - they were always ready to help me with anything, and were even willing to learn how to code just to make things easier for me. Thank you for the fact that when I had classes until 9 PM, I could get to Akademmistechko in 20 minutes instead of an hour and a half, and thank you for always cooking so much and so well every time I came over, as if it was a wedding and not just my last class of the day. I want to thank my brother for always showing up - early or late - even though he wasn't a student here at that moment, just to help or support me.

I want to thank Veronika Anepska for absolutely everything - for always listening to me, for every party, every conversation, every idea, for believing in each other and supporting each other. Together we built discipline - not just opening TikTok every day to keep our streak alive, but also learning Italian every day, doing yoga every day, setting goals and reaching them together. I'm really happy that this diploma was made with you. I'm sure we have an amazing future ahead, because we're doing everything to make it happen.

I want to thank Bohdana Seheda for sitting next to me in an extra English class in first year and Maryna Chernetska for the fact that we were brought together in first year by the same problem - neither of us understood anything. Those small things helped me find such great friends.

Thank you to Ivan Suprun - after talking to you, everything always feels lighter. Thank you to Marko Hlibovytskyi for always telling me I would be fine when I felt like giving up, and for helping me install Windows when I needed it.

I really want to thank the EBD27 and BE27, and my Verkhovyna company. Thank you to Daria Kozlova for the Integration parties, for Halloween vampires, and for inviting me in my fourth year to get through the hard winter moments and the power outages at Hotel Verkhovyna together. Thank you to Dmytro Shevchuk, Matvii Onyshchenko, Artem Litskevych, Dmytro Vasylchuk, Oleksandr Siryi - you are the best. Every holiday, every party, every memory with you is one of the best in my life. Thank you for every tennis game, every good lunch, every conversation, You are all absolute legends. Thank you to Oleh Skakun for always cheering me up and for driving me home.

Thank you to my friend from Italy, Gabriel, whom I met just before the full-scale invasion started - for helping me feel more confident, for every Eurovision night, for respecting my family and my country, and for the invitation to Italy. Thank you also to my friend from Spain, Markel - I'm sure we will meet someday too.

Thank you to Polina Mamchur, Sofiia Tyshchenko, Sofiia Krotova, Yaroslava Mala and Kateryna Shashkina for every moment we went through together, for finding something positive even during the worst deadline weeks, and for always helping.

I also want to thank Kateryna Pyvovarova for our conversation on the 5th floor of KSE - it was you who came up with the Soulmate feature idea, and I'm really glad that conversation happened.

Thank you to absolutely everyone who came into my life during these years and made it the best it could be - in such difficult times.

Abstract

The abstract serves as a concise summary of your entire thesis, encapsulating key elements on a single page such as:
- *General background information*
- *Objective(s)*
- *Approach and method*
- *Conclusions*

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1 Introduction

Your introduction serves to introduce the topic of your Bachelor thesis and to arouse the reader's curiosity with an overview. Why it is important and how it is structured, we explain here.

You can consider an introduction as a teaser for your bachelor thesis. You arouse interest and give a foretaste by presenting your motivation, your method and the state of research in your introduction.

Convince your examiners already in the introduction that your Bachelor thesis will be exciting. If your professor starts reading your thesis with anticipation and interest, the chances of getting good grades are higher.

Pay particular attention to the following in your introduction:

- ***Introduce the topic*** *- What characterizes the topic?*
- ***Introduce the goal*** *- What do you want to achieve with your thesis?*
- ***Make the reader curious*** *- What motivates the reader to read on?*
- ***Describe the relevance*** *- Why is this bachelor thesis scientifically relevant?*

The introduction should have the following content:

- ***Initial situation presentation of the topic*** *- You introduce the topic with an exciting 'bait'. You provide initial information on the topic and the object of research and explain the current state of research.*
- ***Relevance of the topic motivation*** *- You justify the relevance of your topic (scientifically) and place it in the context of your field. In addition, it is often required that you disclose your personal motivation.*
- ***Objectives*** *- Your introduction should clearly state what the goal of your paper is and what outcome you hope to achieve upon completion of the bachelor thesis.*
- ***Method*** *- You explain the approach and justify the choice of method.*
- ***Structure of the Bachelor's thesis*** *- Finally, you give the reader a general overview of your Bachelor's thesis by explaining the structure, showing the red thread and how the research question is answered.*

Welcome to the template's introductory chapter! Instead of boring you with lorem ipsum, here's a quick guide to what you can do in Typst and, more specifically, in this template.

Need more? Check out the Guide to Typst.

1.1 Basic markup

Typst lets you create bold, italic, or monospaced text with ease. You can also sprinkle in equations like $e^{i\pi} + 1 = 0$ or even inline code like `fn main() { println!("Hello, World!") }`. And because life is better in color: pink, blue, yellow, orange, green, and more! Boldly colorize!

You can also write numbered or unnumbered lists:

- First item
- Second item
  1. First Subitem

2. Second Subitem
- Third item

Need equations? Sure! They look great as blocks too:

$$\sin(x) = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \ldots = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)!} x^{2n+1}$$

1.2 Images

As they say, a picture is worth a thousand words. Let's add one:

Figure 2: Project logo

1.3 Tables

Tables are great for organizing data. From simple to complex, Typst handles them all:

Name	Age	City
Albert Einstein	25	Bern
Marie Curie	22	Paris
Isaac Newton	30	London

Table 1: Simple table

[31:27]			[24:20]	[19:15]	[14:12]	[11:7]	[6:0]
funct5	aq	rl	rs2	rs1	funct3	rd	opcode
5			5	5	3	5	7

Table 2: Complex table

1.4 Boxes

Highlight key points with these fun boxes (and more):

Infobox: For highlighting information.

Ideabox: Share a brilliant idea.

Warningbox: Proceed with caution!

Firebox: This is 🔥 !

Rocketbox: Shoot for the moon!

Todobox: Just do it!

TODO

Personnal todo before marking this thesis as final

1.5 Citations, Acronyms and Glossary

Add citations with `@` like [1] or [1, p.7ff] (stored in **`/tail/bibliography.bib`**).

Acronym terms like Infotronics (IT) expand on first use and abbreviate after IT. Glossary items such as Rust Programming Language (Rust) can also be used to show their description as such: Rust is a modern systems programming language focused on safety, speed, and concurrency. It prevents common programming errors such as null pointer dereferencing and data races at compile time, making it a preferred choice for performance-critical applications.. Acronyms and glossary entries auto-generate at the document's end (defined in **`/tail/glossary.typ`**).

1.6 Code

Besides writing inline code as such `fn main() { println!("Hello World") }` you can also write code blocks like this:

```
fn main() {
  let ship = Starship::new("USS Rustacean", (0.0, 0.0, 0.0));
  let destination = (42.0, 13.0, 7.0);
  let warp = ship.optimal_warp(ship.distance_to(destination));

  println!("🖐 {} traveling to {:?} at Warp {:.2}", ship.name, destination,
warp);
  if warp <= 9.0 {
    println!("✨ Warp engaged!");
  } else {
    println!("⚠ Warp failed!");
  }
}
```

Listing 1: First part of the USS-Rustacean code

or directly from a file

```
struct Starship {
  name: String,
  position: (f64, f64, f64),
}

impl Starship {
  fn new(name: &str, position: (f64, f64, f64)) -> Self {
    Self {
      name: name.into(),
      position,
    }
  }
  fn distance_to(&self, dest: (f64, f64, f64)) -> f64 {
    ((dest.0 - self.position.0).powi(2)
      + (dest.1 - self.position.1).powi(2)
      + (dest.2 - self.position.2).powi(2))
    .sqrt()
  }
  fn optimal_warp(&self, distance: f64) -> f64 {
    (distance / 10.0).sqrt().min(9.0)
  }
}
```

Listing 2: Second part of the USS-Rustacean code from **`/resources/code/uss-rustacean.rs`**

1.7 Context Problem

Haute École d'Ingénierie (HEI) Rust Rust programs

[1], [1, p.7ff]

```
fn main() {
  println!("Hello World!");
}
```

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1.8 Objectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

1.9 Structure of this report

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2 Research

In this section, you perform a State of the Art investigation of your domain to understand what's been built, how it works, and where it falls short. You'll survey existing solutions—tools, frameworks and approaches—then compare them against your project's requirements to justify why a new tool (and its specific feature set) is needed.

A focused Domain Area Research unfolds in these steps:

1. *Clarify your research questions and functional requirements*
2. *Collect and review candidate solutions*
3. *Evaluate each alternative's strengths, weaknesses, maturity and architectural style*
4. *Group similar approaches into meaningful categories (for example, monolithic vs. microservice, commercial vs. open-source)*
5. *Pinpoint gaps or missing features that motivate your own design*

This process carries your initial concept through systematic research all the way to a clear, actionable set of requirements for your proposed tool.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Contents

2.1 Section 1 10
2.2 Section 2 10
2.3 Conclusion 10

2.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

2.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

3 Design

3.1 Architecture Overview and Requirements Alignment

The MovieCrush architecture was designed to meet the specific system requirements identified during the domain analysis. The main focus is on personalizing recommendations, storing user data and supporting social features between users. Special attention was given to movie content specifics: fast movie search, making recommendations based on watch history, and handling many ratings and interactions.

The system is built as a modular monolith for the main backend, plus a separate machine learning microservice for recommendations. This approach keeps the main business logic clear and easy for one developer to maintain, while moving heavy ML calculations into an independent service that can be updated and scaled separately.

3.1.1 Functional Requirements

3.1.1.1 Personalized Movie Recommendations

The recommendation system is built as a hybrid three-step pipeline that combines collaborative filtering, content selection, and re-ranking using a large language model:

- **Collaborative Filtering (ALS)**. A separate Python microservice on FastAPI trains an ALS model from the `implicit` library on a user-movie interaction matrix (views, ratings, favorites). The idea is simple: if two users rated many of the same movies in a similar way, they likely have similar tastes. So movies one user liked can be interesting to the other - even if they haven't seen them yet.
- **Content Backup (DISCOVER)**. For variety and for new users, the system adds candidates based on genres and preferences using the TMDB Discover API.
- **Re-ranking via Gemini**. The combined list of candidates (ALS + DISCOVER) is sent to the Google Gemini 2.5 Flash large language model, which picks the 15 best options and assigns a category to each: strong_match (clearly matches the taste), diversity (different from usual but may be liked) or hidden_gem (little-known but worth attention). Results are cached for 24 hours to avoid calling the model on every request.

For new users who don't yet have enough watch history, a separate cold start mechanism works: based on actors and genres chosen during onboarding, the system makes initial recommendations through TMDB until enough data accumulates for ALS to work.

3.1.1.2 Social Features and "Soulmate" Matching

The social system works on a follower model between users. In addition, there is a "movie soulmate" search feature - the user with the most similar taste. Similarity is calculated as a weighted combination of five metrics: cosine similarity of ratings (weight 0.45), overlap of watched movies (0.22), shared favorite actors (0.16), mood similarity (0.11), and agreement on negative ratings (0.06). The final similarity_score is stored in a PostgreSQL table, allowing quick sorting and candidate matching. Search is only performed among users who have explicitly agreed to take part in this feature.

3.1.1.3 Comprehensive Rating System

The rating architecture supports both a simple overall score (scale 1-10) and a detailed aspect rating: direction, acting, script, music, and visual effects (each on a scale 1-5). This allows leaving either a quick rating or a detailed review.

3.1.1.4 Search and Content Discovery

Information about movies and series is obtained through the catalog module, which calls the TMDB API. The module uses lazy loading: detailed movie data is fetched only when the user opens its page. At the same time, metadata is stored in a PostgreSQL table (tmdb_media_cache), so the next request goes to the local database instead of TMDB - faster and cheaper. This cache is also used when generating annual Wrapped statistics.

3.1.1.5 User Engagement Features

The MovieCrush Wrapped module collects user actions over a year (views, ratings, moods, favorite actors) and builds personalized statistics. Calculations rely on cached movie metadata, so dozens of requests to TMDB are not needed at generation time.

3.1.2 Non-Functional Requirements

- **Performance**. Critical read operations are optimized with two-level caching: movie metadata is stored in a PostgreSQL table (tmdb_media_cache), and frequently used data is kept in the application's in-memory LRU cache. This reduces load on both the external TMDB API and the database. Gemini re-ranking results are cached for 24 hours.
- **Data Consistency**. For critical operations (ratings, lists, social connections), ACID transactions in PostgreSQL are used. Integrity is also ensured by foreign keys and constraints at the database schema level.
- **Scalability**. The modular monolith with clear boundaries between modules (auth, profile, movies, soulmate, recommendations, and so on) allows individual parts to be extracted into independent services in the future. The ML component is already a separate service, deployed and scaled independently.
- **Security**. The system uses JWT authentication with refresh token rotation. Passwords are hashed with the bcrypt algorithm. The API is protected by security HTTP headers (helmet), rate limiting, and CORS policies. The mobile app communicates with the backend over HTTPS - Render automatically provides a TLS certificate for all public services.
- **Reliability**. Services are deployed in the cloud as Docker containers on the Render platform. The database is Render Postgres (managed PostgreSQL), located in the same infrastructure as the backend: this removes an extra network hop and improves connection stability. The recommendation model is scheduled for retraining on a cron job to stay up-to-date.

3.2 System Architecture and Major Decisions

3.2.1 Modular Monolith Architecture Decision

Decision: Implement a modular monolith architecture with clear boundaries between domains instead of a microservices architecture.

Reasoning: Given the team size (one developer), project timeline, and current system scale, a modular monolith provides several benefits:

- **Development efficiency.** One developer does not waste time setting up inter-service communication, distributed queues, and separate deployments for each service. This allows faster feature implementation and easier system debugging.
- **Data consistency.** Critical operations, such as marking a movie as watched, update several tables at once: user_movie_actions, user_lists, and counters in users. All of this runs inside a single ACID PostgreSQL transaction - no complex distributed transaction logic between services.
- **Deployment simplicity.** One Docker container - one deployment. The CI/CD pipeline stays simple and predictable.
- **Future scalability.** The system modules have clear boundaries (auth, profile, movies, lists, follows, soulmate, recommendations, wrapped, etc.) so if needed, a single module can be extracted into an independent service without rewriting the whole application. The resource-heavy ML component is already separated this way - as a separate CF service on Python.

Trade-offs considered: A microservices architecture provides independent scaling for each component and technological flexibility. However, its complexity and operational costs greatly outweigh the benefits for the current project with one developer.

3.3 System Context and External Interactions

The system context diagram shows where MovieCrush fits in the wider ecosystem of external services and user interactions. The platform acts as a central hub that connects users with movie rating and discovery features, integrating with external services for content, artificial intelligence, cloud hosting, and email delivery.

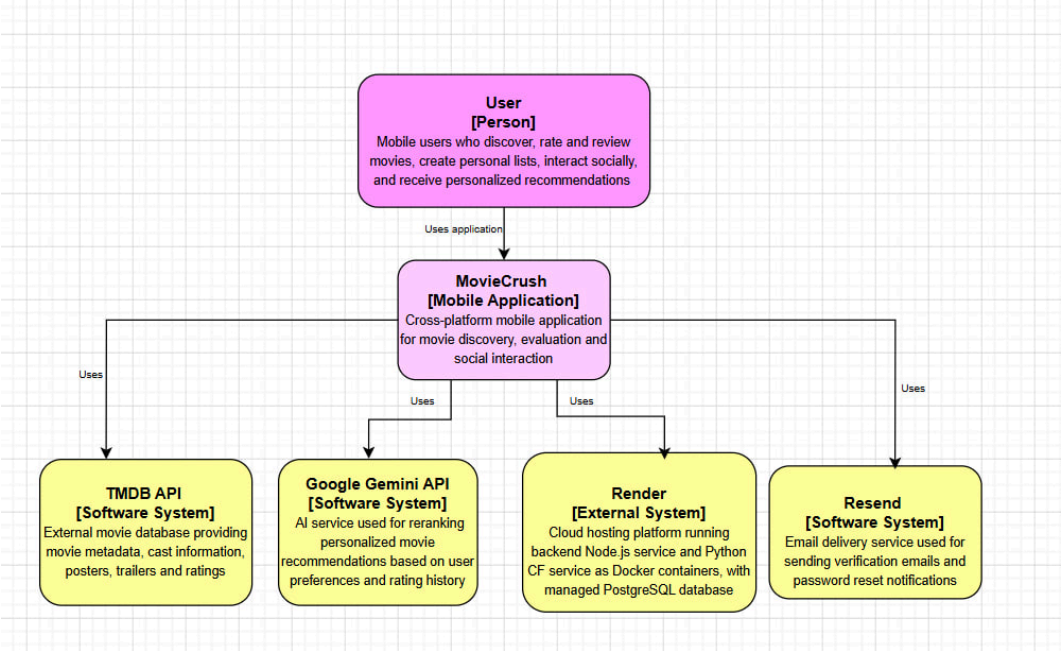

Figure 3: System Context Diagram - MovieCrush Platform Ecosystem. Available here

3.3.1 Main External Integrations

3.3.1.1 TMDB API

Provides the full catalog of movies and series: metadata, actor information, posters, backdrops, trailers, and external ratings. To stay within free tier limits, two caching levels are implemented: an in-memory LRU cache with a 5-minute TTL for hot requests, and permanent metadata storage in a PostgreSQL table (tmdb_media_cache) for data that does not change often (title, genres, cast, release year).

3.3.1.2 Google Gemini 2.5 Flash

Used for re-ranking recommendations. It receives a candidate list from the ALS model and TMDB Discover, analyzes the user's taste based on their ratings, and picks the 15 best options, assigning a category to each. Results are cached in the database for 24 hours to avoid calling the model on every request.

3.3.1.3 Resend

A transactional email service used to send email verification letters during registration and password recovery emails.

3.3.1.4 Render

A cloud platform where two Docker containers are deployed: the main Node.js backend and the Python CF service (ALS recommendations). Both services are in the same Render infrastructure, and the database (Render Postgres - managed PostgreSQL) is located there as well, which removes extra network delays between the backend and the database. Render automatically provides HTTPS for all public endpoints.

3.3.2 Mobile Users

The main users of the platform, who interact with MovieCrush through a mobile app for Android and iOS built on React Native/Expo. The app is built as an APK for Android using EAS Build, the iOS version is cross-platform compatible but requires an Apple Developer account for public distribution.

Users can: search and discover new movies and series, leave detailed ratings, create their own lists, follow other users and find their movie soulmate, and receive personalized recommendations.

3.4 Container Architecture and Service Decomposition

The container diagram shows how MovieCrush is organized on the inside: the mobile app, the main backend as a modular monolith, a separate machine learning service, the database, and external integrations.

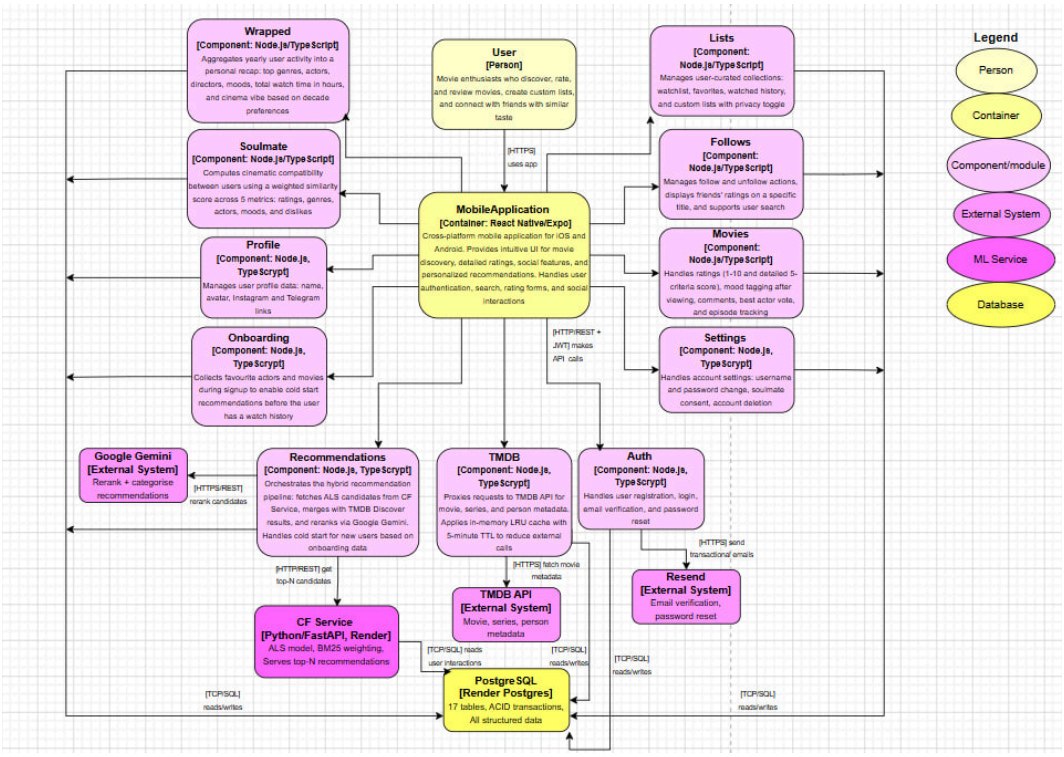

Figure 4: Container Diagram - MovieCrush Architecture and Data Flows. Available here

3.4.1 Backend Modules

Module	Responsibility	Key Functions
Auth	Authentication	Registration, login, email verification, password reset
Profile	User profile	View and edit name, avatar, Instagram/ Telegram links

Module	Responsibility	Key Functions
Settings	Configuration	Username, password, soulmate consent, account deletion
Movies	Movies and series	Ratings (1-10 + 5-criteria detailed score), mood tagging, comments, best actor vote, episode tracking
Lists	Lists	Watchlist, favorites, watched, custom lists with privacy toggle
Follows	Social connections	Follow/unfollow, view friends' ratings on a specific title, user search
Soulmate	Soulmate search	Similarity score across 5 metrics, result storage, on-demand recompute
Recommendations	Recommendations	Hybrid pipeline: ALS via CF service + Gemini reranking, cold start for new users
TMDB	Catalog	Movie/series/person details, search, trailers, cast via TMDB API with LRU cache
Onboarding	Onboarding	Favourite actors and movies selection during signup for cold start
Wrapped	Yearly statistics	top genres, actors, directors, moods, total watch time, cinema vibe

Table 3: Backend modules and their responsibilities

3.4.2 Data Storage Layers

Storage	Purpose
PostgreSQL (Render Postgres)	Main database - 17 tables, ACID transactions. Stores everything: accounts, ratings, lists, social connections, TMDB metadata cache, recommendation results
LRU in-memory cache	Caches TMDB API responses at the process level, TTL 5 minutes, maximum 500 entries
tmdb_media_cache (PostgreSQL)	Permanent storage of movie metadata (title, genres, cast, year). Used by Wrapped and recommendations
user_ai_recommendations (PostgreSQL)	Cache for Gemini re-ranking results, TTL 24 hours

Table 4: Data storage layers and their purposes

3.5 Technology Stack Selection

3.5.1 Frontend - Mobile Application

Technology	Decision	Justification
React Native	Cross-platform mobile framework	One codebase for both iOS and Android. As a solo developer, building two separate native apps was not realistic. Expo adds helpful build tools and allows updates without going through app stores.
Expo + EAS Build	Build and distribution	EAS Build creates the APK in the cloud, so no local Mac is needed. For Android testing, the APK is shared directly without a Play Store account.
TypeScript	Type system	Catches type errors at compile time. With many API response shapes across the app, having typed interfaces makes it much easier to find mismatches between the mobile client and the backend.
React Navigation	Navigation	The standard navigation library for React Native. Works well on both platforms and handles all the screen changes in the app.

Table 5: Frontend technology decisions

3.5.2 Backend

Technology	Decision	Justification
Node.js	Runtime	JavaScript on the server means the same language as the mobile client, which reduces switching between languages. The non-blocking I/O model also works well for an API that makes many requests at the same time to TMDB, Gemini, and the CF service.
TypeScript	Type system	Strict mode is turned on across the whole codebase. Typed request and response interfaces make code changes safer and reduce the chance of runtime errors in production.

Technology	Decision	Justification
Express.js	HTTP framework	Lightweight and simple, which fits the modular monolith structure well. Each module sets up its own router, so the codebase stays organised without the extra weight of a bigger framework.
pg	PostgreSQL client	Raw SQL gives full control over queries, which matters for complex joins in the soulmate algorithm and the Wrapped calculations.
bcryptjs	Password hashing	Standard for storing passwords securely. Passwords are never stored as plain text.
jsonwebtoken	Authentication	JWT with refresh token rotation. Stateless tokens work well for a mobile client that cannot keep server-side sessions.
helmet	Security headers	Sets HTTP security headers with a single middleware call.
express-rate-limit	Rate limiting	Limits each IP to 200 requests per minute on all `/api/*` routes.
lru-cache	In-memory caching	Saves TMDB API responses in process memory with a 5-minute lifetime and a maximum of 500 entries. No separate caching service like Redis is needed.
resend	Email delivery	Handles verification and password reset emails. Simpler to set up than self-hosted email and has better delivery success.
Jest + ts-jest	Unit testing	Used to test the soulmate similarity algorithm, ALS service logic, Wrapped calculations, auth validators, episode helpers, and TMDB helpers.

Table 6: Backend technology decisions

3.5.3 CF Service - Collaborative Filtering

Technology	Decision	Justification
Python	Language	The ML ecosystem in Python is much more mature than in Node.js. Running the model in a separate Python service keeps the backend clean and free of heavy libraries.

Technology	Decision	Justification
FastAPI	HTTP framework	Async-friendly and minimal. Faster to write than Flask for a service that only exposes a few endpoints.
implicit	ALS library	Provides a fast ALS implementation for feedback data. BM25 weighting is applied to the user-item matrix before training to account for different interaction strengths - watched, rated, favourited.
scipy	Sparse matrices	The user-item matrix is mostly empty by nature. scipy's CSR format stores only non-zero entries, which keeps training and prediction memory-efficient.
scikit-learn + matplotlib	Evaluation	Used offline to measure model quality and to tune settings before deploying the trained model.
psycopg2	PostgreSQL client	Reads user interaction data directly from the shared PostgreSQL database to build the training matrix.

Table 7: CF service technology decisions

3.5.4 Database

Technology	Decision	Justification
PostgreSQL	Primary database	ACID-compliant relational database. The data model - users, ratings, lists, social connections, soulmate scores, recommendation cache - is relational with many foreign key constraints and multi-table transactions. JSONB columns are used to store cast data in `tmdb_media_cache`.

Table 8: Database technology decisions

3.5.5 Infrastructure and Deployment

Technology	Decision	Justification
Docker	Containerisation	Both the backend and the CF service are packaged as Docker images using multi-stage builds. A build stage compiles TypeScript or installs Python dependencies, and a small runtime stage contains only what is needed to run. This makes deployments

Technology	Decision	Justification
		work the same way across different environments.
Render	Cloud hosting	Both Docker containers are deployed on Render as separate web services that can be updated independently. AWS was the original plan, but billing issues with an existing account made it too risky to rely on. Render gives Docker-based deployment with automatic HTTPS and no infrastructure management.
Render Postgres	Managed PostgreSQL	At first the database was hosted on Neon, a separate managed PostgreSQL provider. The problem was that every query from the backend had to travel over the public internet to reach it, which added unnecessary delay and meant the database was exposed outside the private network. Moving to Render Postgres placed the database in the same infrastructure as the backend, which removed the cross-internet round trip and kept the database off the public internet. Render Postgres also includes automatic daily backups.
GitHub Actions	CI/CD	The pipeline runs on every push to **`main`** and feature branches. It runs a TypeScript type check, the Jest unit test suite, builds the production bundle, checks that the CF service imports correctly, and builds both Docker images to make sure everything compiles before deployment.

Table 9: Infrastructure and deployment decisions

3.6 Cross-cutting Concerns

3.6.1 Security

3.6.1.1 Authentication and Authorisation

The system uses JWT authentication with two separate tokens. After a successful login, the user receives an access token that lives for 15 minutes and a refresh token that lives for 30 days. The access token is sent in the Authorization header as **`Bearer <token>`** with every API request. The authMiddleware checks the token signature and makes sure the token type is access, since refresh tokens are rejected on protected endpoints. All backend modules apply this middleware before handling any request.

3.6.1.2 Passwords

Passwords are hashed using bcryptjs before saving. Plain text passwords are never stored anywhere in the system.

3.6.1.3 Data Transfer Security

All communication between the mobile app and the backend happens over HTTPS. Render automatically provides and renews TLS certificates for all public services, so manual certificate management is not needed.

3.6.1.4 API Protection

The backend uses helmet to set security HTTP headers on every response, and express-rate-limit to limit each IP address to 200 requests per minute on all `/api/*` routes.

3.6.2 Monitoring and Observability

The current monitoring approach is deliberately simple and matches the scale of the project.

3.6.2.1 Logging

The backend logs key events to standard output using console.log and console.error - startup confirmation, database connection status, and errors. Render automatically captures this output and shows it in the service dashboard. The CF service uses the same approach with print output for model training progress and recommendation requests.

3.6.2.2 Health Check

The backend has a `/health` endpoint that returns `{ status: "ok" }`. Render uses this to monitor service availability and automatically restarts the container if it stops responding.

3.6.2.3 Possible Improvements

For a production system with higher load, the next steps would be structured JSON logging with a library like Winston, centralised log aggregation, and alerts for error rates and response time. In the current version of the project, this was outside the scope.

3.6.3 Error Handling

All database operations are wrapped in try/catch blocks. For critical operations that update several tables at once - for example, marking a movie as watched, which updates user_movie_actions, user_lists, and counters in users - explicit transactions with BEGIN, COMMIT, and ROLLBACK are used. If any step inside a transaction fails with an error, all changes are undone and the database stays in a consistent state.

3.6.4 Data Consistency

The database schema ensures data integrity at the PostgreSQL level. All tables with user data have foreign keys with ON DELETE CASCADE - for example, ratings, lists, comments, follows, and soulmate matches are automatically deleted if the user itself is deleted. This makes sure no records are left in the database without a link to an existing account.

3.7 Database Design

The database schema was designed before implementation, with each table carefully planned in terms of field types, constraints, and storage requirements. For each table, the maximum size per row was calculated in bytes, and then projected across three growth scenarios to estimate total storage needs.

3.7.1 Schema Overview

The final implementation contains 17 tables. The schema went through several revisions during development - some tables from the original plan were removed (for example, a separate **`movies`** table was replaced by the **`tmdb_media_cache`** approach), and new ones were added based on emerging requirements.

3.7.2 Storage Estimation Methodology

For each table, the row size was calculated by summing the maximum byte size of each field. Three projection scenarios were used:

Scenario	MAU	Registered users (× 2.2)
Conservative	30 000	66 000
Realistic	100 000	220 000
Optimistic	250 000	550 000

Table 10: Growth scenarios used for storage estimation

The 2.2 coefficient converts MAU to total registered users - it accounts for the typical ratio between accumulated and active audiences in social apps, based on the assumption that around 45-55% of registered users remain active after 12 months.

For the content catalog, the Pareto principle (80/20 rule) was applied: since roughly 20% of titles receive 80% of user interactions, only 294,000 out of 1.26 million TMDB movies were considered realistic candidates for active caching. This avoided overestimating storage for the media cache table.

User behaviour assumptions were also applied per table - for example, comments were estimated using a 90/9/1 split: 90% of users write zero comments, 9% write 1-5, and 1% write 10-50+, giving an average of 0.47 comments per user.

3.7.3 Key Design Decisions

Dual ID system in users table. Each user has an internal **`BIGINT id`** used for all foreign key relationships and joins, and a separate **`CHAR(36) uuid`** exposed in public-facing APIs. This prevents internal IDs from being enumerable in API responses.

Counter columns in users table. Fields like **`movies_watched`**, **`followers_count`**, and **`friends_count`** are stored as denormalised counters updated by application logic rather than computed on every query. This trades a small write overhead for significantly faster profile reads.

JSONB for cast data. The **`top_cast`** column in **`tmdb_media_cache`** stores the top 10 actors as a JSONB array rather than a separate join table. Since cast data is always read as a

unit and never queried individually, this avoids an unnecessary join on every movie detail request.

ON DELETE CASCADE on all user-owned data. Every table that stores user content - ratings, lists, comments, follows, soulmate matches - has a foreign key to `users(id)` with `ON DELETE CASCADE`. Deleting a user account automatically removes all associated data without requiring application-level cleanup logic.

The full storage calculations and per-field justifications were documented during the discovery phase and are available at: Data Modelling Spreadsheet.

3.8 User Growth Projections

To estimate system load, realistic user growth targets were defined first. Benchmarking against competitors was considered but rejected because Letterboxd has 17 million monthly users after 13 years on the market, which is not a useful baseline for a new product.

Instead, growth projections were built around four key drivers specific to MovieCrush:

- **Wrapped virality** - the strongest driver, similar to how Spotify Wrapped generates millions of social media posts every December
- **Soulmate feature** - social motivation to invite friends and find matches
- **AI recommendations** - a useful feature that drives retention
- **Paid marketing** - eventual budget for user acquisition

Three scenarios were defined:

Scenario	Year 1 MAU	Year 2 MAU	Year 3 MAU
Conservative	30,000	100,000	250,000
Realistic	100,000	500,000	1,000,000
Optimistic	250,000	1,200,000	2,500,000

Table 11: MAU growth projections across three scenarios

The realistic scenario is grounded in four concrete arguments:

1. Ukraine has 30 million people. Capturing 0.3% of the population gives 100,000 users - a realistic target for a quality niche app.
2. If 20% of 100,000 users share their Wrapped report on Instagram (20,000 posts) and each brings 2 new users, that adds 40,000 users through virality alone.
3. Expanding to the US and Europe is a planned next step after the Ukrainian market is established. These markets are significantly larger and would make 500 000+ users achievable in year 2-3 with the right positioning.
4. Comparable social apps at launch - Goodreads, early Duolingo - showed similar growth patterns in their first two years.

DAU is estimated at 20% of MAU, which is a standard ratio for social and entertainment apps. These numbers feed directly into the load estimation in the next section.

3.9 Capacity Planning and Load Estimation

Before making architectural decisions, a load estimation was done to understand how many requests the system would need to handle at different growth stages. The full calculations are available at: Load Estimation Spreadsheet.

One user makes roughly 81 requests per day across all actions: opening the app, searching for films, viewing movie pages, rating, commenting, and browsing recommendations. Using 20% of MAU as DAU, requests per second (RPS) were calculated for three scenarios:

Scenario	MAU	DAU	RPS
Conservative - Year 1	30,000	6,000	5.56
Conservative - Year 2	100,000	20,000	18.52
Realistic - Year 1	100,000	20,000	18.52
Realistic - Year 2	500,000	100,000	92.59
Optimistic - Year 1	250,000	50,000	46.3
Optimistic - Year 2	1,200,000	240,000	222.22

Table 12: RPS estimates across growth scenarios

Request types follow a read-heavy pattern: 70% GET (browsing movies, profiles, recommendations), 20% POST (ratings, comments, lists), 8% PUT (profile and settings updates), and 2% DELETE.

The key findings from this analysis directly shaped architectural decisions:

- **5.56 RPS (conservative Year 1)** - a single server handles this comfortably. This confirmed that starting with a modular monolith on a single Render instance was the right call, with no need for load balancing at launch.
- **92.59 RPS (realistic Year 2)** - at this level, caching becomes critical. This justified the two-level caching strategy: in-memory LRU for TMDB responses and PostgreSQL tables for recommendation results.
- **222+ RPS (optimistic)** - would require horizontal scaling and potentially extracting high-traffic modules into separate services. The modular monolith structure was chosen specifically to make this transition possible without rewriting the whole system.

4 Implementation

In this section you translate your component-level designs into working code and systems. Focus on the C4 Component layer and on the details needed to show how your design was realized. Include only the most important code snippets that illustrate key patterns or algorithms, rather than full listings.

1. *Describe the development methodology (for example, Agile or test-driven development) used to guide your implementation*
2. *Explain any prototyping or iterative strategies you applied to refine components before full-scale coding*
3. *Summarize coding standards, naming conventions and architectural patterns followed in your codebase*
4. *Present critical code snippets or configuration templates that highlight how core components were implemented (for example, key classes, interfaces or algorithms)*
5. *Detail your testing approach and quality assurance measures (unit tests, integration tests, coverage metrics)*
6. *Note any performance optimizations or profiling results for components that were bottlenecks*
7. *Outline your deployment and configuration management process for component artifacts (containerization, CI/CD pipelines)*
8. *Highlight documentation deliverables (API references, inline comments, architecture decision records) that support future maintenance*

This section demonstrates how each component specification becomes actual, maintainable code—closing the loop from design to implementation.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Contents

4.1 Section 1 26
4.2 Section 2 26
4.3 Conclusion 26

4.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

4.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

4.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

5 Validation

Validation (Requirements Verification and Testing)

In this section you demonstrate how your implementation satisfies the initial requirements through clear testing methods and concise examples—suitable for a bachelor-level project:

1. *Restate each key functional and non-functional requirement from your Analysis and Design sections*
2. *Describe the testing approach for each requirement (for example, unit tests, manual acceptance checks or scenario walkthroughs)*
3. *Provide concrete test cases or usage examples that show how you verify each requirement in practice*
4. *Summarize actual versus expected outcomes, indicating pass/fail status for each test*
5. *Include brief snippets of test code or sample console outputs to illustrate your procedures*
6. *Note any gaps or deviations and suggest simple fixes or areas for future improvement*
7. *If a feature wasn't intended for specific scenarios (e.g. high-load), omit unrealistic stress tests and clearly document its current limitations*

This focused structure ties every requirement directly to validation results, using examples and methods you can realistically carry out at the bachelor level.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Contents

5.1 Section 1 28
5.2 Section 2 28
5.3 Conclusion 28

5.1 Section 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

5.2 Section 2

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

5.3 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

6 Conclusion

Conclusion

In this final section you bring together your work and reflect on its impact. Keep it concise, restating key points without introducing new information:

1. *Project Summary: Briefly recap objectives, methodology and principal results*
2. *Alignment with Objectives: Discuss how outcomes meet initial goals, referencing requirements and design aims*
3. *Lessons Learned and Challenges: Note any obstacles and how they informed improvements*
4. *Limitations: Acknowledge features or scenarios beyond this scope and clearly state current system boundaries*
5. *Future Work: Suggest practical enhancements or research directions building on your findings*

Avoid introducing new concepts here; refer readers to the Discussion for deeper analysis.

6.1 Project summary

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

6.2 Comparison with the initial objectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

6.3 Encountered difficulties

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

6.4 Future perspectives

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Glossary

Programming Language

Rust **– Rust Programming Language**: Rust is a modern systems programming language focused on safety, speed, and concurrency. It prevents common programming errors such as null pointer dereferencing and data races at compile time, making it a preferred choice for performance-critical applications. 6, 7

University

HEI **– Haute École d'Ingénierie** 7

IT **– Infotronics** 6

Bibliography

[1] S. Zahno *et al.*, "Dynamic Project Planning with Digital Twin," *Frontiers in Manufacturing Technology*, vol. 3, May 2023, doi: 10.3389/fmtec.2023.1009633.

A Appendix

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.